

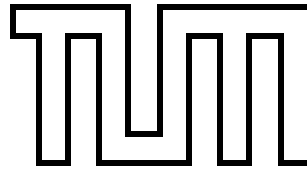
SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Informatics

**Formal Verification of Earley's Parsing
Algorithm, Including Running Time
Analysis**

Alexander Haberl



SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY - INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Informatics

**Formal Verification of Earley's Parsing Algorithm,
Including Running Time Analysis**

**Formale Verifikation von Earleys Parser
Algorithmus mit Laufzeitanalyse**

Author: Alexander Haberl
Examiner: Prof. Tobias Nipkow, Ph.D.
Supervisor: Prof. Tobias Nipkow, Ph.D.
Date: 11.05.2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, 11.05.2026

Alexander Haberl

Abstract

In this thesis, we refine the prior work done by Nipkow and Rau on the Earley algorithm into an executable list-based recognizer and formalize both a set-based and a list-based parser. We prove correctness and termination of the recognizer and parsers for all epsilon-free grammars and discuss ways to tackle those as well. In the list-based algorithms, we implement a data structure proposed by Earley to facilitate fast item lookup, to verify the running time bounds put forth by Earley. We introduce a parameter for the random access time of the partitions in this data structure and conduct a parameterized asymptotic running time analysis, verifying the cubic time bound under the assumption of constant access time. Our list-based implementations achieve only $O(n^4)$ running time instead.

Zusammenfassung

In dieser Arbeit verfeinern wir die Ergebnisse von Nipkow und Rau zu Earleys Parsing-Algorithmus zu einer auf Listen basierenden ausführbaren Implementierung und erweitern diese zu einem auf Sets und einem auf Listen basierenden Parser. Wir beweisen Korrektheit und Termination für alle Epsilon-freien Grammatiken und geben Vorschläge, wie dieses Problem gelöst werden kann. Für die listenbasierten Algorithmen implementieren wir eine von Earley vorgeschlagene Datenstruktur, welche schnelles item nachschlagen erlaubt, um die Laufzeiten, die Earley angibt, zu verifizieren. Wir schaffen einen Parameter für die zufällige Zugriffszeit auf die Partitionen in dieser Datenstruktur und bauen darauf eine asymptotische Laufzeitanalyse auf. Unter der Annahme konstanter Zugriffszeiten bestätigen wir die kubische Laufzeit. Unsere auf Listen basierende Implementierung erzielt nur eine $O(n^4)$ Laufzeit.

Contents

Abstract	iv
Zusammenfassung	v
1 Introduction	1
2 Background	2
2.1 Context-Free Grammars	2
2.2 Parse Trees	3
3 Earley Recognizer Formalization	5
3.1 Inductive definition	7
3.2 Towards executability	8
3.3 Workset closure	9
3.4 Epsilon treatment	11
4 Executable List-based Version	12
4.1 Space Analysis	15
4.2 Efficient Itemlist Datastructure for running time guarantees	15
5 Running Time Analysis	19
5.1 Efficient Itemlist	20
5.2 Earley algorithm	20
6 Expansion to Parse Trees	23
6.1 List-based parser	25
6.2 Runing Time Analysis of Parsing Algorithm	27
7 Related Work	29
8 Conclusion	30
Bibliography	31

1 Introduction

In informatics, Context-Free Grammars are an important tool for describing many different things, for example, the syntax of programming languages, as well as file and binary data formats. As such, the correct parsing of these is important for accurate program compilation and even for security in parsing binary formats. Because of this, many different parsing algorithms exist for context-free languages, but only a few have been formally proven. For programming languages, there exist many parsers that specialize in performance, some of which are also verified. These often only work on a smaller subclass of context-free languages. One powerful all-purpose parser was proposed by Earley in 1970 [3], guaranteeing a worst-case cubic running time bound and better bounds for specific subclasses of grammars. The only formal verification of this algorithm we know of is the one we built on by Nipkow and Rau [9], which itself was built on a paper by Jones [5]. These formalizations did not yet yield executable parsers, as they used abstract set types and therefore did not formally analyze their running time bounds. Our thesis seeks to close this gap by implementing an executable, list-based version of the recognizer and parser and formally proving the cubic time bound put forth by Earley.

In this thesis, we start by providing a background on context-free grammars and parse trees in Isabelle in [chapter 2](#). We then recap Earley’s algorithm [3] and the work of Nipkow and Rau [9], which serves as the basis for our implementations in [chapter 3](#). Our contributions start by refining the recognizer into a list-based executable version using the data structure proposed by Earley, and proving its correctness and termination in [chapter 4](#). In [chapter 5](#), we conduct a parameterized asymptotic analysis of the running time of our algorithm. We then augment this algorithm to function as a parser in [chapter 6](#), implementing both a set- and a list-based version, and perform a similar running time analysis to that in the recognizer case in [section 6.2](#). Lastly, we provide an overview of related work, including other formally verified parsers, in [chapter 7](#).

2 Background

2.1 Context-Free Grammars

Languages are sets of words, which are strings made up of terminal symbols. The whole set of terminal symbols is called the alphabet Σ . One way to define what words are in a language is context-free grammars (CFGs), which introduce new non-terminal symbols and productions that formalize how to transform a string of non-terminal and terminal symbols to derive words step by step, starting from a single non-terminal. A production expresses one such possible step, where you replace one non-terminal symbol by a string of non-terminal and terminal symbols. The Language of a CFG is then the set of strings containing only terminal symbols that can be reached from the starting non-terminal, following the modifications described by the productions. Strings of non-terminals and terminals are called sentential forms, and strings of only terminal symbols are called sentences or words.

Formally, in Isabelle, strings are represented by lists, and so to represent sentential forms, non-terminals and terminals are grouped together into the wrapper symbol datatype *sym*, which can represent either a terminal or non-terminal symbol.

datatype (*'n*, *'t*) *sym* = *Nt* *'n* | *Tm* *'t*

Productions (A, α) are a pair of a non-terminal A and a sentential α , where the non-terminal is called the left-hand side (lhs) and the sentential is called the right-hand side (rhs) of the production. In text, we will write productions like this ($A \rightarrow \alpha$). Sets of productions have type *Prods* or *prods* when stored as lists. A set of productions P then defines the so-called derivation relation $P \vdash u \Rightarrow v$, which is formally defined as follows:

derive :: (*'n*, *'t*) *Prods* $\Rightarrow$ (*'n*, *'t*) *sym list* $\Rightarrow$ (*'n*, *'t*) *sym list* $\Rightarrow$ *bool*
($P \vdash u \Rightarrow v$) = ($\exists A \beta x y. u = x @ [Nt A] @ y \wedge v = x @ \beta @ y \wedge (A, \beta) \in P$)

This represents one derivation step as described informally above. Context-free grammars are made up of a set of productions P and a non-terminal as a starting symbol, commonly named S . The Language of a CFG is then defined in terms of the transitive closure of the derivation relation $P \vdash u \Rightarrow^* v$. Specifically, it is the set of sentences that are reachable from the starting non-terminal S .

datatype (*'n*, *'t*) *Cfg* = *Cfg* ((*'n*, *'t*) *Prods*) *'n*

Lang :: (*'n*, *'t*) *Prods* $\Rightarrow$ *'n* $\Rightarrow$ *'t list set*

Lang $P A = \{w \mid P \vdash [Nt A] \Rightarrow^* map Tm w\}$

LangS :: (*'n*, *'t*) *Cfg* $\Rightarrow$ *'t list set*

LangS $\equiv \lambda G. Lang (Prods G) (Start G)$

A CFG implicitly defines the sets of terminals and non-terminals by which appear in its productions.

$Nts_syms :: ('n, 't) \text{ sym list} \Rightarrow 'n \text{ set}$
 $Nts_syms \ w = \{A \mid Nt \ A \in \text{set } w\}$
 $Nts :: ('n, 't) \text{ Prods} \Rightarrow 'n \text{ set}$
 $Nts \ P = (\bigcup_{(A, w) \in P} \{A\} \cup Nts_syms \ w)$
 $Tms_syms :: ('n, 't) \text{ sym list} \Rightarrow 't \text{ set}$
 $Tms_syms \ w = \{a \mid Tm \ a \in \text{set } w\}$
 $Tms :: ('n, 't) \text{ Prods} \Rightarrow 't \text{ set}$
 $Tms \ P = (\bigcup_{(A, w) \in P} Tms_syms \ w)$

There are many interesting computational questions concerning languages. One very natural question is, given a word w , is it in the language given by a CFG? This problem is called recognition. A natural continuation is asking about the derivation of such a word if it is in the language, which is called parsing.

2.2 Parse Trees

Parse trees are a way to encode a derivation as a tree structure. The inner nodes represent one production, and the leaves are symbols. In Isabelle, there exists the parse tree datatype *tree*:

datatype $('n, 't) \text{ tree} = \text{Sym } (('n, 't) \text{ sym}) \mid \text{Rule } 'n \ (('n, 't) \text{ tree list})$

A production, also called a *Rule* in the context of parse trees, consists of a non-terminal (the lhs) and a list of child trees, representing the symbols of the right-hand side in order. So for every child, it has to hold that either it is a leaf of the corresponding symbol, or it is a rule with the corresponding non-terminal as its left-hand side. Thus, for a given set of productions, we can define the following predicate that a parse tree has to fulfill:

$\text{parse_tree} :: ('n, 't) \text{ Prods} \Rightarrow ('n, 't) \text{ tree} \Rightarrow \text{bool}$
 $\text{parse_tree} _ (\text{Sym } _) = \text{True}$
 $\text{parse_tree} \ P (\text{Rule } A \ ts) =$
 $((\forall t \in \text{set } ts. \text{parse_tree } P \ t) \wedge (A, \text{map } \text{root } ts) \in P)$

Then there exist two access functions for the parse tree. The projection *root* returns the non-terminal of a rule or the symbol at the root of the tree, and *fringe* returns the concatenation of the leaves of a parse tree.

$\text{root} :: ('n, 't) \text{ tree} \Rightarrow ('n, 't) \text{ sym}$
 $\text{root } (\text{Sym } s) = s$
 $\text{root } (\text{Rule } A \ _) = Nt \ A$
 $\text{fringe} :: ('n, 't) \text{ tree} \Rightarrow ('n, 't) \text{ sym list}$
 $\text{fringe } (\text{Sym } s) = [s]$
 $\text{fringe } (\text{Rule } uu \ ts) = \text{concat } (\text{map } \text{fringe } ts)$

We use these to define the *valid_parse_tree* predicate that checks for a CFG, consisting of productions P and starting symbol S , and a word w , that the parse tree is correct and represents the derivation showing that w is a word of the CFG's language.

$$\begin{aligned} \text{valid_parse_tree} &:: ('n, 'a) \text{ Prods} \Rightarrow ('n, 'a) \text{ sym list} \Rightarrow 'n \Rightarrow ('n, 'a) \text{ tree} \Rightarrow \text{bool} \\ \text{valid_parse_tree } P \ w \ S \ t &\equiv \text{parse_tree } P \ t \wedge \text{root } t = \text{Nt } S \wedge \text{fringe } t = w \end{aligned}$$

And finally, we prove that parse trees represent derivations by showing that for any derivation, there exists a valid parse tree and that the existence of a valid parse tree implies that there exists a derivation for its word.

$$\begin{aligned} P \vdash [\text{Nt } A] \Rightarrow^* w &\implies \exists t. \text{parse_tree } P \ t \wedge \text{fringe } t = w \wedge \text{root } t = \text{Nt } A \\ \text{parse_tree } P \ t &\implies P \vdash [\text{root } t] \Rightarrow^* \text{fringe } t \end{aligned}$$

While for every parse tree there exists a single derivation it represents, a derivation may have multiple different parse trees. The existence of different parse trees for a word is a property of the grammar, and so we call grammars in which every word has exactly one parse tree *unambiguous*.

$$\text{unambiguous} :: ('n, 'a) \text{ Cfg} \Rightarrow \text{bool}$$

$$\text{Earley_Gw_eps } ps \implies$$

$$\text{unambiguous } g \equiv$$

$$\forall w \in \text{Lang } S \ g.$$

$$\forall t1 \ t2.$$

$$\text{valid_parse_tree } (\text{Prods } g) \ (\text{map } \text{Tm } w) \ (\text{Start } g) \ t1 \wedge$$

$$\text{valid_parse_tree } (\text{Prods } g) \ (\text{map } \text{Tm } w) \ (\text{Start } g) \ t2 \longrightarrow$$

$$t1 = t2$$

3 Earley Recognizer Formalization

In this thesis, we formalize a recognizer first proposed by Earley [3]. Our contributions are based on prior work by Nipkow and Rau [9]. We present their work in a condensed form here, as it serves as an important basis for further proofs. Our contributions start in [chapter 4](#) and include a discussion of how to handle sets of productions that can derive the empty word in [section 3.4](#).

We fix, for the following chapter, a word w and a CFG C that induces the language L . The CFG will have the starting symbol S and either set P or list ps of productions. Earley's recognizer builds sets of items for each index of w , where an item in one of these sets represents that a certain substring of w has been recognized. These items consist of a production and two natural numbers called dot and from.

datatype $(\text{'n}, \text{'a}) \text{ item} = \text{Item } (\text{prod}: (\text{'n}, \text{'a}) \text{ prod}) (\text{dot} : \text{nat}) (\text{from} : \text{nat})$

We will write an item $\text{Item } p \ d \ f$ like this $(A \rightarrow \alpha \bullet \beta, f)$, where the production is $(A, \alpha\beta)$ and $|\alpha| = d$. The idea behind the recognizer is that an item $\text{Item } p \ k \ l$ in set $\mathcal{S}_i$ symbolizes that we have recognized the substring of w starting at index l and ending at index i , and that it can be derived from the partial right-hand side of p that is left of the dot at position k . We call this property soundness.

Definition 1 (Soundness). $\text{Item } p \ k \ l \in \mathcal{S}_i \implies P \vdash \text{slice } 0 \ k \ (\text{rhs } p) \Rightarrow^* \text{slice } l \ i \ w$.

The function $\text{slice } i \ k \ s$ returns the substring starting at index i and ending at index $k - 1$. It is equivalent to $\text{take } k \ (\text{drop } i \ s)$.

For better readability, we define some helper functions:

```
is_complete :: ('n, 'a) item ⇒ bool
is_complete x = (|rhs (prod x)| ≤ dot x)

next_symbol :: ('n, 'a) item ⇒ ('n, 'a) sym option
next_symbol x =
  (if is_complete x then None else Some (rhs (prod x) ! dot x))

next_sym_Nt ≡ λx A. next_symbol x = Some (Nt A)

mv_dot :: ('n, 'a) item ⇒ ('n, 'a) item
mv_dot x ≡ Item (prod x) (dot x + 1) (from x)
```

The predicate is_complete checks whether the dot is at the end of the production, next_symbol returns an option of the symbol after the dot, next_sym_Nt checks if the next symbol is a specific non-terminal, and mv_dot moves the dot over the next symbol.

We also define a predicate to check whether an item is well-formed in the context of the

recognizer algorithm, meaning we check what items could appear in a set $\mathcal{S}_k$. We check that the production is part of the grammar, that the dot is a valid index into the right-hand side of the production, that the from-value is less than or equal to k , and that k is less than or equal to the length of w .

$$\begin{aligned} wf_item &:: ('n, 'a) \text{ item} \Rightarrow \text{nat} \Rightarrow \text{bool} \\ wf_item \ x \ k &= (\text{prod } x \in P \wedge \text{dot } x \leq |\text{rhs } (\text{prod } x)| \wedge \text{from } x \leq k \wedge k \leq |w|) \end{aligned}$$

Earley then defines three operations for the recognizer algorithm that generate items for these sets in an inductive manner. These operations are called *Predict*, *Complete*, and *Scan*. If we have an item in set $\mathcal{S}_i$ with its dot in front of a non-terminal A , we predict. So we start recognizing a new a substring of w , starting at index i , from the non-terminal A .

$$\text{PREDICT: } \frac{x \in \mathcal{S}_i \quad \text{next_sym_Nt } x \ A \quad p \in P \quad \text{lhs } p = A}{\text{Item } p \ 0 \ i \in \mathcal{S}_i}$$

Completion occurs when we have fully recognized an item $B \rightarrow \alpha\bullet, f$, meaning its dot is at the end of the right-hand side of the production. In this case, we have recognized a substring of w starting at f . This recognition started with the non-terminal B , the one on the production's left-hand side. So we can extend the recognition of substrings up to f ($P \vdash u \Rightarrow^* \text{slice } k \ f \ w$) with this substring recognition as follows: $P \vdash u @ [\text{Nt } B] \Rightarrow^* \text{slice } k \ i \ w$. In terms of items, this means we take an item that recognizes a substring up to f from set $\mathcal{S}_f$ and extend its recognition if the symbol after the dot is the non-terminal B .

$$\text{COMPLETE: } \frac{\begin{array}{l} x \in \mathcal{S}_i \quad \text{is_complete } x \quad \text{from } x = f \\ y \in \mathcal{S}_f \quad \text{next_sym_Nt } y \ (\text{lhs } (\text{prod } x)) \end{array}}{\text{mv_dot } y \in \mathcal{S}_i}$$

The last operation *Scan* occurs when the next symbol of an item is a terminal. In this case, we can advance recognition of the substring if the next symbol in w is that terminal. In this case, we add these items to the next set $\mathcal{S}_{i+1}$, since the length of the recognized substring of w has increased by one.

$$\text{SCAN: } \frac{x \in \mathcal{S}_j \quad j < |w| \quad \text{next_symbol } x = \text{Some } (w ! j)}{\text{mv_dot } x \in \mathcal{S}_{(j+1)}}$$

Lastly, we need a starting point for this inductive definition. For this, we use the result of applying the *Predict* operation to the starting symbol. We will be referring to this as the fourth operation *Init*.

$$\text{INIT: } \frac{p \in P \quad \text{lhs } p = S}{\text{Item } p \ 0 \ 0 \in \mathcal{S}_0}$$

From these inductive definitions, we can derive the following functions that produce sets by applying the operations to a single item. For the definition of the *Complete* set, we get a list Bs of computed bins for which $Bs ! i = \mathcal{S}_i$ holds.

$$\begin{aligned} \text{Predict } it \ i &= \{\text{Item } p \ 0 \ i \mid p \in P \wedge \text{next_sym_Nt } it \ (\text{lhs } p)\} \\ \text{Complete } Bs \ y &= \{\text{mv_dot } it \mid it \in Bs ! \text{from } y \wedge \text{next_sym_Nt } it \ (\text{lhs } (\text{prod } y))\} \\ \text{Scan } \mathcal{S}_i \ i &= \{\text{mv_dot } it \mid it \in \mathcal{S}_i \wedge \text{next_symbol } it = \text{Some } (w ! i)\} \\ \text{Init} &= \{\text{Item } p \ 0 \ 0 \mid p \in P \wedge \text{lhs } p = S\} \end{aligned}$$

We also define a top-level predicate for w , called *accepted*, that returns true if we find a complete item with the start symbol as its left-hand side in the last set $\mathcal{S}_{|w|}$. By soundness, this means that $P \vdash [Nt S] \Rightarrow^* w$ holds.

Definition 2. $accepted = (\exists x \in \mathcal{S}_{|w|}. is_final\ x)$

where $is_final\ x = (lhs\ (prod\ x) = S \wedge from\ x = 0 \wedge is_complete\ x)$

We give an example to better understand and visualize the algorithm: Given a CFG with productions $P = \{A \rightarrow AB, A \rightarrow BA, A \rightarrow a, B \rightarrow b\}$ and starting non-terminal A , we get the following sets for the word "bab".

$\mathcal{S}_0$		$\mathcal{S}_1$		$\mathcal{S}_2$		$\mathcal{S}_3$	
$A \rightarrow \bullet AB$	0	$B \rightarrow b\bullet$	0	$A \rightarrow a\bullet$	1	$B \rightarrow b\bullet$	2
$A \rightarrow \bullet BA$	0	$A \rightarrow B\bullet A$	0	$A \rightarrow BA\bullet$	0	$A \rightarrow AB\bullet$	1
$A \rightarrow \bullet a$	0	$A \rightarrow \bullet AB$	1	$A \rightarrow A\bullet B$	1	$A \rightarrow AB\bullet$	0
$B \rightarrow \bullet b$	0	$A \rightarrow \bullet BA$	1	$A \rightarrow A\bullet B$	0	$A \rightarrow BA\bullet$	0
		$A \rightarrow \bullet a$	1	$B \rightarrow \bullet b$	2	$A \rightarrow A\bullet B$	1
		$B \rightarrow \bullet b$	1			$A \rightarrow A\bullet B$	0
						$B \rightarrow \bullet b$	3

Table 3.1: Example of the sets constructed by the Earley recognizer

3.1 Inductive definition

Towards a formal proof of this recognizer, we define an inductive Earley set, which will be the union of all $\mathcal{S}_i$ where $i \leq |w|$. To differentiate which set each item is in, we will store pairs of and Earley item and a natural number indicating which set it belongs to. The full set is constructed inductively using the three operations and the initial set discussed previously.

Earley :: $((n, a)\ item \times nat)\ set$

For correctness, we need to prove the two directions of soundness and completeness. This means that if the recognizer accepts a word, it is in the language, and vice versa, that if a word is in the language, the recognizer accepts it. Soundness is quite easy to prove for this definition. We only need to prove that the soundness predicate holds for every item added to this set, which is done by proving that all operations preserve soundness and showing that the initial set is sound. From these individual proofs, overall soundness follows:

Theorem 1. $accepted \implies P \vdash [Nt S] \Rightarrow^* w$

To prove completeness, we show that if there is an item in the Earley set and the rest of the item's right-hand side after the dot can derive a substring of w following the substring already derived by the item, then the complete item will also be in the set.

Lemma 1. $j \leq k \implies k \leq |w| \implies x = \text{Item } (A, \gamma) \text{ } d \text{ } i \implies (x, j) \in \text{Earley}$
 $\implies P \vdash \beta \text{ } x \Rightarrow (n) \text{ slice } j \text{ } k \text{ } w \implies (\text{Item } (A, \gamma) \text{ } |\gamma| \text{ } i, k) \in \text{Earley}$

This is proven using complete induction on the length of the derivation and an induction on the length of the rest of the right-hand side of the item. From this, full completeness follows rather easily, as the initial set contains all possible incomplete items after one step of derivation. Therefore, if the entire word is derivable, one of these items will be complete in the final Earley-set $\mathcal{S}_{|w|}$, and so the word is accepted by the recognizer.

Theorem 2. $P \vdash [\text{Nt } S] \Rightarrow^* w \implies \text{accepted}$

3.2 Towards executability

While this theoretical Earley recognizer is well-suited for proves, it is poorly suited for execution due to its inductive definition. Instead, the iterative set computation suggested by Earley is better suited. Looking at the four operations of the Earley algorithm, we see that only *Predict* and *Complete* add items to the current set, while *Init* is the starting set, and *Scan* adds items to the next set, resulting in the starting set of the next set. Every operation also depends only on items in sets with an index lower than or equal to the current set's index, allowing us to compute the sets in order. So we can split the algorithm into two steps: first, compute the starting set for set $\mathcal{S}_i$ using *Init* or *Scan*, and then compute the closure of this set under *Predict* and *Complete*. We can repeat this until we have computed all sets..

Going forward, we will call item sets *bins*, to which we extend the well-formedness predicate. The algorithm will follow this general form:

```
bins :: nat => ('n, 'a) item set list
bins 0 = [Close [] Init]
bins (k + 1) = (let Bs = bins k in Bs @ [Close Bs (Scan (last Bs) k)])
```

It computes a list of bins as described before, using *Close* to compute the closure. This way, we can refine the closure algorithm step by step while the underlying structure of the algorithm remains unchanged. From now on, *Bs* will always refer to the previously computed list of bins.

To prove that this new algorithm computes the same sets as the fully inductive definition, we first use an inductive set closure. It takes a starting set *B* - either a result of *Init* or *Scan* - and the previously computed list of bins *Bs*, and returns the closure *C* under *Predict* and *Complete*.

```
Close :: ('n, 'a) item set list => ('n, 'a) item set => ('n, 'a) item set
Close Bs B =
Init : x ∈ B ⟹ x ∈ Close Bs B
Predict : [x ∈ Close Bs B; x' ∈ Predict x |Bs|] ⟹ x' ∈ Close Bs B
Complete : y ∈ Close Bs B ⟹ is_complete y ⟹ from y = |Bs| ⟶ x ∈ Close Bs B
           ⟹ from y < |Bs| ⟶ x ∈ Bs ! from y ⟹ next_sym_Nt x (lhs (prod y))
           ⟹ mv_dot x ∈ Close Bs B
```

We then prove that the *bins* method using *Close* is equivalent to the original Earley implementation.

Theorem 3. $i \leq |w| \implies \text{bins } |w| \text{ ! } i = (\mathcal{S}_i)$

Here, the function $\mathcal{S}$ collects all items of the fully inductive Earley set that are in the i -th set.

We are working towards a one-pass closure algorithm that only needs to consider every element exactly once. This is possible if the items added by *Predict* and *Complete* do not depend on the items in the current set closure we are computing. This is the case for the *Predict* operation, which only depends on the set of productions. For the *Complete* operation, it is only the case if the complete item's from-value is the index of a previous bin, which is true if the production has a length greater than zero. Handling the *Complete* operation for length-zero productions is not straightforward, so we require the grammar to be epsilon-free.

Definition 3. $\text{Eps_free} :: ('a \times 'b \text{ list}) \text{ set} \Rightarrow \text{bool}$

$\text{eps_free } ps = (\forall (lhs, rhs) \in P. rhs \neq [])$

This way, we simplify the algorithm. We also provide options for handling non-epsilon-free grammars in 3.4. So under this assumption, we create another closure, *Close1*, which works the same as *Close*, but the *Complete* operation is simplified, as we assume the grammar to be epsilon-free:

$\text{Complete} : y \in \text{Close1 } Bs \ B \implies \text{is_complete } y \implies x \in Bs \text{ ! from } y$
 $\implies \text{next_sym_Nt } x \ (lhs \ (prod \ y)) \implies \text{mv_dot } x \in \text{Close1 } Bs \ B$

We prove equivalence to *Close* under the assumption of epsilon-freeness.

Lemma 2. $\text{Eps_free } P \implies \text{wf_bins1 } Bs \implies \text{wf_bin1 } B \ |Bs|$

$\implies \text{Close1 } Bs \ B = \text{Close } Bs \ B$

Unless specified otherwise, we will assume that the grammar is epsilon-free from here on out and drop the *eps_free* ps assumption. The next step towards executability is replacing the inductive closures with a one-pass worklist algorithm.

3.3 Workset closure

We keep track of a pair of sets, the workset B and the accumulator C . The workset comprises items in the closure that have not yet been considered, while the accumulator keeps track of all previously considered items. So at any step, $B \cup C$ are all items we know to be in the closure, and it should hold that $B \cap C = \emptyset$. One step of this worklist algorithm then looks like this:

- choose an item from B to be considered $b \in B$
- compute the set of items D added by b under *Predict* or *Complete*

- move b from B to C and add D to B , while making sure that $B \cap C = \emptyset$ by the end

The last step we do as follows $(B \cup D - (C \cup \{b\}), C \cup \{b\})$. To formalize this approach, we define a step relation that relates any two such set pairs of workset and accumulator that differ by one step.

Definition 4. *Close2* ::

$(\text{'n'}, \text{'a'})$ item set list

$\Rightarrow (\text{'n'}, \text{'a'})$ item set $\times$ $(\text{'n'}, \text{'a'})$ item set

$\Rightarrow (\text{'n'}, \text{'a'})$ item set $\times$ $(\text{'n'}, \text{'a'})$ item set $\Rightarrow \text{bool}$

Predict : $x \in B \Rightarrow \neg \text{is_complete } x$

$\Rightarrow Bs \vdash (B, C) \rightarrow (B \cup \text{Predict } x \mid Bs \mid - (C \cup \{x\}), \{x\} \cup C)$

Complete : $x \in B \Rightarrow \text{is_complete } x$

$\Rightarrow Bs \vdash (B, C) \rightarrow (B \cup \text{Complete } Bs \ x - (C \cup \{x\}), C \cup \{x\})$

The closure is then given by the set C when repeated steps result in $B = \emptyset$. This will always be the case, as the set of valid Earley items is finite. More on this in 4.1. Formally, this means we are looking for the accumulator C of a set pair $(\emptyset, C)$ that is reachable from $(B, \emptyset)$ under the step relation. Since this formalization is non-deterministic in the order that items are chosen from the workset, we require the Hilbert choice operator for the formalization to get a random but distinct set C , if it exists. In reality, this set is unique and always exists.

Definition 5. *close2* :: $(\text{'n'}, \text{'a'})$ item set list $\Rightarrow (\text{'n'}, \text{'a'})$ item set $\Rightarrow (\text{'n'}, \text{'a'})$ item set

close2 $Bs \ B = (\text{SOME } C. Bs \vdash (B, \emptyset) \rightarrow^* (\emptyset, C))$

We again prove equivalence of *close2* and *Close1*.

Lemma 3. $\llbracket \text{wf_bins1 } Bs; \text{wf_bin1 } B \mid Bs \mid \rrbracket \Rightarrow \text{close2 } Bs \ B = \text{Close1 } Bs \ B$

For this proof, we need two other lemmas. First, that *close2* terminates, and second that the result is equal to the set obtained by *Close1*. We will first focus on the set equality. The direction that *close2* is a subset of *Close1* is simple, since that the result after one step is always a subset of the final result of *Close1*. The other direction is a bit trickier. We do a proof by induction on the items in *Close1* and have to check three cases. First, the initial items in B are also in the result of *close2*, which is easily apparent because any item in B is moved to C at some point. Second, for the induction step, we have to prove that for any item x with $x \in \text{Close1 } Bs \ B$ and $x \in C$, all items obtained by the operations *Predict* or *Complete* on x also end up in C . This proof boils down to the observation that if x ends up in C , there must exist a step in which it is picked from B and moved to C . In this step, all items added to *Close1* by x are also added to B in the *Close2* step and therefore end up in C eventually.

For termination, we define a well-founded potential function that decreases for every step. It is a lexicographic ordering primarily on the number of well-formed items not in the worklist or accumulator, and secondarily on the size of the worklist. The number of items not in the worklist or accumulator decreases whenever an item is added through *Predict* or

Complete. If no new items are added, the worklist's size decreases by 1, since one item is always moved to the accumulator.

After proving that the step relation always decreases this potential, we can prove termination of the algorithm and finish the equivalence proof of *close2* and *Close1*.

3.4 Epsilon treatment

In this section, we discuss different ways of handling grammars that contain epsilon productions. As a first solution, we can leverage the fact that there exist algorithms that transform any CFG C into another CFG C' with $\text{LangS } C' = \text{LangS } C - \{\varepsilon\}$. We can now write a wrapper to the Earley recognizer, which runs the recognizer on C' if $w \neq \varepsilon$, and checks if $\varepsilon \in \text{LangS } C$ otherwise. This has already been formalized in the theory `Epsilon_Elimination`¹ by Martin Stimpfle, where *eps_elim* transforms a grammar into one without epsilon productions and the predicate *Nullable*, which returns if a non-terminal can derive the empty word. While this is a simple solution, it has the drawback that the resulting epsilon free grammar may be much larger than the original.

Another approach that does not require rewriting the grammar would be to modify the closure algorithm and enable it to handle epsilon productions. For this, we suggest changing the *Predict* operation and consequently the Init set. If there are epsilon productions in a grammar, multiple Nts may derive the empty word $P \vdash [\text{Nt } B] \Rightarrow^* []$ even over multiple steps.

Whenever an item $A \rightarrow \alpha B \bullet \beta, k$ is added by the *Complete* operation on an item with a from-value equal to the current set, the item $(A \rightarrow \alpha \bullet B \beta, k)$ must also be in the current set. In this case, we can add this item to the set of items added by the *Predict* operation:

$$\text{predict}' \text{ it } i = \text{predict } \text{it } i \cup (\text{if } P \vdash [\text{the } (\text{next_symbol } \text{it})] \Rightarrow^* [] \text{ then } \{\text{mv_dot } \text{it}\} \text{ else } \emptyset)$$

And this will solve our problem, since the operation depends only on the set of productions. To make this efficient, it would be better to compute the set of Nts that can derive the empty word once and then do a lookup as the if condition.

¹https://www.isa-afp.org/thys/Context_Free_Grammar/Epsilon_Elimination.html

4 Executable List-based Version

We can now implement a list-based version. For this, we write list-based implementations of the Earley operations and implement the repetition of the closure algorithm using a functional while loop. This version will not be enough to prove the cubic running time guarantees put forth by Earley. To that end, we introduce a data structure to efficiently look up items in [section 4.2](#). We first start by implementing the four Earley operations for lists using the map and filter functions:

$Init_L :: ('n, 'a) \text{ item list}$

$Init_L = \text{map } (\lambda p. \text{Item } p \ 0 \ 0) \ (\text{filter } (\lambda p. \text{lhs } p = S) \ ps)$

$Predict_L :: ('n, 'a) \text{ item} \Rightarrow \text{nat} \Rightarrow ('n, 'a) \text{ item list}$

$Predict_L \ it \ k = \text{map } (\lambda p. \text{Item } p \ 0 \ k) \ (\text{filter } (\lambda p. \text{next_sym_Nt } it \ (\text{lhs } p)) \ ps)$

$Complete_L :: ('n, 'a) \text{ item list list} \Rightarrow ('n, 'a) \text{ item} \Rightarrow ('n, 'a) \text{ item list}$

$Complete_L \ Bs \ it = \text{map } mv_dot \ (\text{filter } (\lambda b. \text{next_sym_Nt } b \ (\text{lhs } (prod \ it))) \ (Bs \ ! \ \text{from } it))$

$Scan_L :: ('n, 'a) \text{ item list} \Rightarrow \text{nat} \Rightarrow ('n, 'a) \text{ item list}$

$Scan_L \ B \ k = (\text{let } x = \text{Some } (w \ ! \ k) \text{ in } \text{map } mv_dot \ (\text{filter } (\lambda b. \text{next_symbol } b = x) \ B))$

We then prove equivalence with the set-based versions defined earlier.

$\text{set } Init_L = \text{Init} \quad \text{set } (Predict_L \ it \ k) = Predict \ it \ k$

$\llbracket is_complete \ it; \text{wf_item1 } it \ |Bs| \rrbracket \implies \text{set } (Complete_L \ Bs \ it) = Complete \ (\text{map set } Bs) \ it$

$k < |w| \implies \text{set } (Scan_L \ B \ k) = Scan \ (\text{set } B) \ k$

Using the *Predict* and *Complete* operations in conjunction with the set operations on lists, we implement *step_fun_L*, which computes one step of the closure and thus fulfills *Close2*. For brevity, we introduce the *Close2_L* relation, which is a wrapper for the *Close2* relation that converts lists to sets.

$step_fun_L ::$

$(('n, 'a) \text{ item list list})$

$\Rightarrow (('n, 'a) \text{ item list} \times ('n, 'a) \text{ item list})$

$\Rightarrow (('n, 'a) \text{ item list} \times ('n, 'a) \text{ item list})$

$step_fun_L \ Bs \ (b \ \# \ bs, \ cs) =$

$(\text{let } nexts = \text{if } is_complete \ b \text{ then } Complete_L \ Bs \ b \text{ else } Predict_L \ b \ |Bs|$

$\text{in } (nexts \cup_L (b \ \# \ bs) -_L (\{b\} \cup_L cs), \{b\} \cup_L cs))$

$Close2_L ::$

$(\text{'n'}, \text{'a'}) \text{ item list list}$

$\Rightarrow (\text{'n'}, \text{'a'}) \text{ item list} \times (\text{'n'}, \text{'a'}) \text{ item list}$

$\Rightarrow (\text{'n'}, \text{'a'}) \text{ item list} \times (\text{'n'}, \text{'a'}) \text{ item list} \Rightarrow \text{bool}$

$Close2_L \text{ Bs } (bs, cs) (bs', cs') =$

$(\text{map set Bs} \vdash (\text{set bs, set cs}) \rightarrow (\text{set bs'}, \text{set cs'}))$

This implementation satisfies the $Close2$ relation, because the list-based Earley operations are equivalent to the set-based functions.

$B \neq [] \Rightarrow wf_bins1 (\text{map set Bs}) \Rightarrow wf_bin1 (\text{set B}) |Bs|$

$\Rightarrow \text{step_fun}_L \text{ Bs } (B, C) = (B', C') \Rightarrow Close2_L \text{ Bs } (B, C) (B', C')$

We implement the closure using $while_option$ in conjunction with $step_fun_L$. The while option construction takes a condition c , a body function b , and a starting value s , and returns the first value that violates c and is a result of repeatedly applying b to s , if such a result exists. It can be proven that this is equivalent to the following executable version.

$while_option \ c \ b \ s = (\text{if } c \ s \ \text{then } while_option \ c \ b \ (b \ s) \ \text{else } Some \ s)$

Proving predicates about the result of $while_option$, assuming that a result exists, requires the predicate to be preserved across applications of the body's function and be valid for the starting value. Then a simple proof by induction shows that the predicate must hold for the result as well.

$\llbracket \bigwedge s. \llbracket P \ s; \ c \ s \rrbracket \Rightarrow P \ (b \ s); \ while_option \ c \ b \ s = Some \ t; \ P \ s \rrbracket \Rightarrow P \ t$

The implementation of the closure then boils down to repeatedly applying $step_fun_L$ until the worklist is empty, using $while_option$.

$steps_L ::$

$(\text{'n'}, \text{'a'}) \text{ item list list}$

$\Rightarrow (\text{'n'}, \text{'a'}) \text{ item list} \times (\text{'n'}, \text{'a'}) \text{ item list}$

$\Rightarrow ((\text{'n'}, \text{'a'}) \text{ item list} \times (\text{'n'}, \text{'a'}) \text{ item list}) \text{ option}$

$steps_L \text{ Bs } BC = while_option \ (\lambda(B', C'). B' \neq []) \ (step_fun_L \text{ Bs}) \ BC$

To prove equality to the work set-based closure $close2$, we prove that the result of the $while_option$ loop is part of the transitive closure of $Close2$ and that the while loop terminates.

The equivalence of the two relations $step_fun_L$ and $Close2$ is a simple consequence of the equivalence of the Earley and list operations to their set counterparts. Proving that the loop result is in the transitive closure uses the proof technique described above. We simplify the proof here:

Proof. We start by constructing the predicate we want to prove, that the result of the loop (B', C') is in the transitive closure starting from the initial lists (B, C) :

$$\lambda(B', C'). (Close2_L Bs)^{**} (B, C) (B', C')$$

Now we have to prove two things: first, that the proposition holds for the starting value, and second, that it holds after the loop body, $step_fun_L$, is applied.

1) $(Close2_L Bs)^{**} (B, C) (B, C)$ holds, as this is the reflective case of the transitive closure.

2) We can assume that the proposition holds for (B', C') and have to show that it also holds for $step_fun_L Bs (B', C')$. From the fact that $step_fun_L$ fulfills the relation, we get $Close2_L Bs (B', C') (step_fun_L Bs (B', C'))$.

Then $(Close2_L Bs)^{**} (B, C) (step_fun_L Bs (B', C'))$ also holds, following from the transitive property of the transitive closure. $\square$

Since the fact that $step_fun_L$ and $Close2_L$ are equivalent requires assumptions about well-formedness and invariants, those also have to be conserved throughout the loop body, making the proposition more complicated, but the underlying argumentation is the same. Using this and the fact that the worklist of the result must be empty by the loop condition, we can show that the accumulator at the end of $steps_L$ is part of the transitive closure of $Close2_L$.

$$\llbracket wf_bins1 (map\ set\ Bs); wf_bin1 (set\ B) |Bs|; steps_L Bs (B, C) = Some (B', C') \rrbracket \implies (Close2_L Bs)^{**} (B, C) ([], C')$$

Termination is proven the same way as in the set version of the algorithm, using the same lexicographical ordering, adapted to work with lists.

From this, the equivalence of $close2_L$ and $close2$ follows.

$$\llbracket wf_bins1 (map\ set\ Bs); wf_bin1 (set\ B) |Bs| \rrbracket \implies set (close2_L Bs B) = close2 (map\ set\ Bs) (set\ B)$$

As a last step, we prove the equivalence of the $bins_L$ and original $bins$ functions. For this, we need to ensure that the bins we compute are well-formed, which requires that the closure result be well-formed. The closure is well-formed, as all items added come from the Earley operations, which we proved to be well-formed.

$$k \leq |w| \implies map\ set (bins_L k) = bins k$$

At this point, we have an executable version of the recognizer, but it does not have the time bounds we want. That is because the list-based difference has a quadratic running time and is called on the full bins for every closure step. Earley introduces a way to speed this up, which we will present after a short space analysis that lays the foundation for the data structure we introduce and the running time analysis.

4.1 Space Analysis

We do a space analysis based on the length of the word, so we assume the number of productions L and the maximum length of a production K are constant. To guarantee that the number of productions is equal to the length of the productions list, ps , we require the list to be duplicate-free, using *distinct* ps .

For the well-formed predicate, we introduced a parameter k that bounds the maximum from-value in a bin. Additionally, we require the items production to be in the set of productions, bounding the number of different field entries by L . The dot has to be a valid index into the rhs of the production, bounding it by K . Thus, the size of a bin, and by extension its space, is bounded by the multiplication of all upper bounds.

$$wf_bin\ xs\ k \implies |xs| \leq L * (K + 1) * (k + 1)$$

This gives us a simple way to bound the size of any set computed by our algorithm, since we have already proven that well-formedness is preserved throughout the algorithm. The size of a list is bounded by the same number, as long as it is distinct, a property guaranteed by list-based set operations and one that must be guaranteed by the implementation of our new data structure.

Since every Earley set bounds the from-values that can appear in it by its index, the size of each set grows linearly with its index. Thus, the overall size is bounded by the sum over all Earley bins, yielding a quadratic total space cost.

$$|S_i| \leq L * (K + 1) * (i + 1)$$

$$|Earley| \leq L * (K + 1) * (|w| + 1)^2$$

Here, the *Earley* set is the one from the formalization that is the union of all Earley bins.

We can also observe that in a well-formed bin, the number of well-formed items for a given from-value is constant ($L * (K + 1)$), a fact that will become relevant for the data structure we build in the next section to prove better running time guarantees for the difference operation.

4.2 Efficient Itemlist Datastructure for running time guarantees

We aim to prove the running time analysis given by Earley [3]. We know from space analysis in [section 4.1](#) that the size of every Earley bin is linearly proportional to its index. Earley proposes a way to construct a set in a time quadratic to its index. This is not possible using standard list-based set operations, as the difference operation has a running time in $O(n^2)$. The solution described by Earley uses a data structure to achieve item lookup in constant time, enabling a linear-time difference operation. This data structure stores the entire bin as multiple partitions, one for every from-value. As we argued in the space analysis, every partition will have a constant size. So if we can access the partitions in constant time, we can fully search any one of them in constant time as well. This makes the constant lookup time possible. As every Earley bin bounds the from-values of items by its index, we can use an array constructed once at the start of each set closure to store the partitions. In our implementation, we only use a list to store the partitions. We could further refine the

data structure by using an array with a functional interface, but this was beyond the scope of this thesis. Instead, we perform a parameterized running time analysis based on the partition access time, allowing us to prove the running time of the same data structure for different partition access times.

This data structure has type *efficientItemList* and will be called *IL* for short. We use it like a normal list to represent the bins. In practice, we implement the *IL* as a normal list and a list of partitions, each stored as a list itself and mirroring the items stored in the list.

datatype (n, a) *efficientItemList* = *IL* $((n, a)$ *item list*) $((n, a)$ *item list list*)

This type has two projection functions, *list* and *froms*, to access the list and the list of partitions. Since the list and the partitions should store the same items, the set represented is simply the set the list represents.

$set_{IL} :: (n, a)$ *efficientItemList* $\Rightarrow (n, a)$ *item set*
 $set_{IL} \text{ il} = set \text{ (list il)}$

The list provides simple constant-time access to an element in the bin, while the partitions are used to access and look up specific elements. So we need an invariant to ensure that the items stored in the list and the partitions are the same, so that the lookup operation using the partitions is equivalent to the membership test of the set, represented by the *IL*.

$inv_{IL} :: (n, a)$ *efficientItemList* $\Rightarrow bool$
 $inv_{IL} \text{ (IL as fs)} =$
 $(0 < |fs| \wedge$
 $(\forall x \in set \text{ as. from } x < |fs|) \wedge$
 $(\forall i < |fs|. set \text{ (fs ! i)} = \{x \in set \text{ as} \mid from \text{ } x = i\}) \wedge$
 $(\forall i < |fs|. distinct \text{ (fs ! i)}) \wedge distinct \text{ as})$

To ensure the validity of the lookup in the partitions, we require that all items can be stored in the reserved partitions, and that an item is in the list if and only if it is present in the partition corresponding to its from-value. We additionally require that both the list and all partitions are distinct. This allows us to use the findings from the space analysis to bound their maximum lengths. Finally, we require the list of partitions to be non-empty for simplicity, as some of the proofs about this invariant are more easily proven automatically if we do not allow an empty list of partitions.

We now define the basic operations for this data structure in accordance with the invariant. These are the *isin_{IL}*, *(#_{IL})*, and *empty_{IL}*, which correspond to the membership test, insertion, and the empty set constructor. These are then used to implement the $(\cup_{IL})$ and $(-_{IL})$ operations, which are equivalent to their set counterparts, and to convert from type *list* to *efficientItemList* using *IL_of_List*.

$isin_{IL} :: (n, a)$ *efficientItemList* $\Rightarrow (n, a)$ *item* $\Rightarrow bool$
 $isin_{IL} \text{ (IL _ fs)} x = isin_list \text{ (fs ! from } x) x$

```

( $\#_{IL}$ ) ::
  ('n, 'a) item  $\Rightarrow$  ('n, 'a) efficientItemList  $\Rightarrow$  ('n, 'a) efficientItemList
x  $\#_{IL}$  IL as fs =
  (if isinIL (IL as fs) x then IL as fs
   else IL (x  $\#$  as) (fs[from x := x  $\#$  fs ! from x]))

empty_froms :: nat  $\Rightarrow$  ('n, 'a) item list list
empty_froms 0 = []
empty_froms (k + 1) = []  $\#$  empty_froms k

empty_IL :: nat  $\Rightarrow$  ('n, 'a) efficientItemList
empty_IL k = IL [] (empty_froms k)

( $\cup_{IL}$ ) ::
  ('n, 'a) item list
 $\Rightarrow$  ('n, 'a) efficientItemList  $\Rightarrow$  ('n, 'a) efficientItemList
[]  $\cup_{IL}$  il = il
a  $\#$  as  $\cup_{IL}$  il = a  $\#_{IL}$  as  $\cup_{IL}$  il

diff_LIL ::
  nat
 $\Rightarrow$  ('n, 'a) item list
 $\Rightarrow$  ('n, 'a) efficientItemList  $\Rightarrow$  ('n, 'a) efficientItemList
diff_LIL k [] = empty_IL k
diff_LIL k (a  $\#$  as) il =
  (if  $\neg$  isinIL il a then a  $\#_{IL}$  diff_LIL k as il else diff_LIL k as il)

( $-_{IL}$ ) ::
  ('n, 'a) efficientItemList
 $\Rightarrow$  ('n, 'a) efficientItemList  $\Rightarrow$  ('n, 'a) efficientItemList
il1  $-_{IL}$  il2 = diff_LIL (|froms il1| - 1) (list il1) il2

IL_of_List :: nat  $\Rightarrow$  ('n, 'a) item list  $\Rightarrow$  ('n, 'a) efficientItemList
IL_of_List k as = as  $\cup_{IL}$  empty_IL k

```

The important details are that the lookup uses the partitions to take constant time in an array implementation, that there is a check for inclusion before inserting an item, to guarantee distinctness, and that we implement the difference operation by inserting into an empty list, making use of the constant-time lookup to check if the item is present in the other *IL*. The natural number given as a parameter to *empty_IL* and *IL_of_List* affects the number of partitions of the resulting item list and thus bounds the highest from-value items

in this list can have. We constructed it so that a bin xs with $wf_bin\ xs\ k$ can be stored in an $empty_{IL}\ k$

All item list functions maintain the invariant and correspond to set functions, provided the items inserted have a from-value that fits within the partitions. Otherwise, the nth and $update$ functions are undefined. In general, we prove given set operation $\bullet$ and the corresponding item list operation $\bullet_{IL}$:

$$\begin{aligned} IL_inv\ il1 &\implies prop\ x \implies IL_inv\ (il1 \bullet_{IL}\ x) \\ IL_inv\ il1 &\implies prop\ x \implies set_IL\ (il1 \bullet_{IL}\ x) = set_IL\ il1 \bullet x \end{aligned}$$

Depending on whether x is a single item or a set of items, the proposition of x ensures that the from-value of x itself or every item in x is lower than the number of partitions in the item list.

We then proceed to replace the lists in the worklist algorithm with these item lists. The functions do not change much, but to differentiate them from the functions that work with pure lists, they have the subscript IL . The proof of correctness then works the same way, with newly added assumptions that must be valid at every step of the algorithm. These are that the invariant of the item list holds for both the worklist and accumulator, and a predicate about the number of partitions in each item list. This last assumption then requires us to also prove that all operations do not change the number of partitions.

We proved, as in the list version, that $bins_{IL}$ computes the same bins as the set-based $bins$ function. And used this equivalence to show that this implementation only recognizes a word if and only if it is in the language.

Theorem 4. $k \leq |w| \implies map\ set\ (bins_{IL}\ k) = bins\ k$

$$earley_recognized \longleftrightarrow P \vdash [Nt\ S] \Rightarrow^* w$$

where $earley_recognized$ checks whether there is a final item in the last bin of $bins_{IL}\ |w|$.

5 Running Time Analysis

We now begin a formal running time analysis to verify the guarantees put forth by Earley. He claims an overall running time of $O(n^3)$. As we mentioned before, we will perform a parameterized running time analysis based on the random-access time of the stored partitions. We specifically assume that both the *nth* and *update* operations are bounded by the same running time function $T_{nth_{IL}}$, which is monotone increasing. As both the *nth* and *update* operations boil down to a random lookup, we bound them by the same function, while monotonicity is needed to aggregate different calls. And both of these assumptions are satisfied by the current list implementation, which has linear running time for the operations, and by the optimal array implementation, which would guarantee constant running time for both operations.

$$T_{list_update} \text{ xs } n \ a \leq n + 1$$

$$T_{nth} \text{ xs } n = n + 1$$

For the analysis, we use the built-in *time_fun* command in Isabelle, which automatically generates a timing function for a given function. The timing functions generated this way count non-primitive function calls. Other operations, like arithmetic computation, variable access, or datatype construction, are assumed to take no time. This approach provides time bounds in the correct O -class, while simplifying the reasoning as much as possible and allowing inductive proofs. A more detailed explanation of the implementation of the automatic timing functions can be found in [1]. The only exception where automation fails is the *steps_{IL}* function, which is currently not supported by *time_fun* due to partial function application in its *while_option* loop.

We give one example of how the generated timing functions look using $(\cup_{IL})$ as an example.

$$T_{\cup_{IL}} :: ('n, 'a) \text{ item list} \Rightarrow ('n, 'a) \text{ efficientItemList} \Rightarrow \text{nat}$$

$$T_{\cup_{IL}} [] = 1$$

$$T_{\cup_{IL}} (a \# as) \ il = T_{\cup_{IL}} \ as \ il + T_{\#_{IL}} \ a \ (as \cup_{IL} \ il) + 1$$

The base case simply returns the given item list resulting in a time of 1 for the application of the function itself. The recursion case is defined as the time taken for the recursive function call plus the insert plus one for the function call itself again.

But as we argued, the list we use for storing the partitions could be implemented as an array with $O(1)$ random-access time, given by the *nth* function. So we make a parameterized running time analysis based on the running time of *nth* for this specific list type. Once we have this, we can argue about the running time, assuming an underlying array or any other data structure. For brevity, we will only show the time bounds without any assumptions,

as they are the same as in the completeness proofs. When discussing time bounds in the text, we assume a constant running time for $T_{nth_{IL}}$ and elaborate on implementations with other access time assumptions only for the final time bounds of the complete algorithm.

5.1 Efficient Itemlist

We start by proving running time bounds for the operations of *efficientItemList*. The basic operations of lookup ($isin_{IL}$) and insertion ($\#_{IL}$) are the only places where the assumed time bounds for nth and $update$ are needed directly. All other functions use $isin_{IL}$ and ($\#_{IL}$).

$$T_{isin_{IL}} \text{ il } x \leq T_{nth_{IL}} (\text{from } x) + L * (K + 1) + 1$$

$$T_{\#_{IL}} x \text{ il} \leq 3 * T_{nth_{IL}} (\text{from } x) + L * (K + 1) + 1$$

And we see that these functions have a constant running time, since they only need to perform a lookup and then search the corresponding partition. Then the running times of the union and difference operations are linear in the length of the list, since they perform either a single lookup or both a lookup and an insert per list element.

$$T_{\cup_{IL}} as \text{ il} \leq |as| * (3 * T_{nth_{IL}} (|\text{froms il}| - 1) + L * (K + 1) + 2) + 1$$

$$T_{-_{IL}} il1 \text{ il2} \leq |list \text{ il1}| * (4 * T_{nth_{IL}} (|\text{froms il1}| - 1) + 2 * L * (K + 1) + 5) + 2 * |\text{froms il1}| + 3$$

The difference operation has an additional linear addend in the number of partitions it requires, stemming from the $empty_{IL}$ time bound, which is linear in the number of partitions it has to construct. In the full algorithm analysis, the list size equals the bin size, and the number of partitions equals the index of the bin currently under construction. As the bin size grows linearly with its index, both addends will also grow linearly with the bin index. Lastly, we have the conversion from lists to item lists, IL_of_List , which is a simple combination of the $empty_{IL}$ and ($\cup_{IL}$) operations and will therefore also be linear.

5.2 Earley algorithm

For the four operations, both the initial set construction and prediction depend only on the number of productions and are therefore constant. The *Complete* and *Scan* operations depend on the size of the set they have to filter, resulting in a time bound that is linear in the set's size. Since every Earley bin is well-formed it has a size linear to its index, and the *Complete* and *Scan* operations are also linear in the index of the set for which the closure is being computed.

$$T_{Init_L} \leq 2 * (L + 1)$$

$$T_{Predict_L} x \text{ k} \leq 2 * (K + 2) * |ps| + 2$$

$$T_{Complete_L} Bs \text{ y} \leq |Bs| + 2 * (K + 2) * |Bs ! \text{ from } y| + 2$$

$$T_{Scan_L} B \text{ k} \leq k + 2 * (K + 2) * |B| + 3$$

Putting these time bounds together with those for the efficient item list, we find that the step_fun_{IL} time bound is also linear in the index of the current bin. This mostly follows directly from the time bounds given and the assumption that the item lists are well-formed and satisfy the invariant inv_{IL} , which requires that the lists be distinct. Distinctness allows us to get length bounds on these lists equal to the sizes of the sets, which are linear in their indices.

We now come to the while loop implementation of the closure for which we had to write a custom timing function, as the built-in timing functions do not work with partial function applications, as we used in the body of the while loop. Our timing function performs the normal computation and maintains a timing accumulator throughout. The accumulator of the result then gives the total running time of the while loop, if it exists.

```

steps_time ::
  ('n, 'a) item list list
  ⇒ ('n, 'a) efficientItemList × ('n, 'a) efficientItemList
  ⇒ nat
  ⇒ (((('n, 'a) efficientItemList × ('n, 'a) efficientItemList) ×
        nat) option
steps_time Bs ils y =
  while_option (λ((il1, il2), k). list il1 ≠ [])
    (λ((il1, il2), k).
      (step_funIL Bs (il1, il2), k + Tstep_funIL Bs (il1, il2)))
    (ils, y)
TstepsIL ::
  ('n, 'a) item list list
  ⇒ ('n, 'a) efficientItemList × ('n, 'a) efficientItemList ⇒ nat
TstepsIL Bs ils = snd (the (steps_time Bs ils 0))

```

Using the argumentation from the termination proof, we know that the size of the list accumulator increases by one every iteration, and so we can bound the entire closure algorithm by the size of the accumulator times the time bound for a single step. This time bound is the predicate we use in our proof about the while loop's result.

$\lambda((il1, il2), k). k \leq |\text{list } il2| * T_{\text{step_fun_bound}}$

Here, $T_{\text{step_fun_bound}}$ is simply an abbreviation for the full numeric time bound. Again, we have to augment this predicate with the other assumptions about well-formedness and invariants, but the core of the proof is about showing that this inequality holds. We can again bound the length of the accumulator by well-formedness and distinctness, so we get a quadratic running time for $T_{\text{steps}_{IL}}$. The time bound for the full closure, close2_{IL} , adds only a linear term for creating an empty item list and is therefore also quadratic.

$$\begin{aligned}
T_{close2_{IL}} \text{ } Bs \text{ } il &\leq (L * (K + 1) * (|Bs| + 1))^2 * \\
&(\gamma * T_{nth_{IL}} |Bs| + 3 * L * (K + 1) + 10 + 2 * (K + 2)) + \\
&L * (K + 1) * (|Bs| + 1) * \\
&(\gamma * T_{nth_{IL}} |Bs| + 2 * L * (K + 1) + \gamma + (K + 1)) + \\
&2 * (|Bs| + 1)
\end{aligned}$$

Lastly, we bound the running time of the *bins_{IL}* function and the top-level predicate, *earley_recognized*. Since *bins_{IL}* computes a linear number of bins, each with a quadratic time bound, the overall time bound is cubic. We show a simplified time bound, but there exist others with tighter constants.

$$\begin{aligned}
C1 &= 28 * (L + 1)^3 * (K + 1)^3 \\
C2 &= 17 * (L + 1)^2 * (K + 1)^2 \\
T_{bins_{IL}} \text{ } k &\leq C1 * (k + 2)^3 + C2 * (k + 2)^3 * T_{nth_{IL}} (k + 1) \\
C1' &= 30 * (L + 1)^3 * (K + 1)^3 \\
T_{earley_recognized} \text{ } ps \text{ } S \text{ } w0 &\leq C1' * (|w| + 2)^3 + C2 * (|w| + 2)^3 * T_{nth_{IL}} (|w| + 1)
\end{aligned}$$

From the total parameterized running time, we see that if we set the random-access time to be constant, we do indeed obtain a cubic running time, as we argued in the text. If we instead take our current Isabelle list-based implementation, we get a running time of $O(n^4)$ instead. This concludes the asymptotic analysis of the recognizer, where we have verified the time bound given by Earley.

6 Expansion to Parse Trees

We now turn this recognizer into a parser. For each item, we save parse trees that make up the derivation that is represented by the item. The way we do this is by creating a pair of an Earley item and a list of parse trees, with access functions *item* and *trees*.

$parseItem :: ('n, 'a) \text{ item} \times ('n, 'a) \text{ tree list}$

The intention is that the item represents the derivation of a substring of w from the symbols to the left of the dot, and that the list of trees stores this derivation by having one child parse tree for every symbol up to the dot, holding part of the derivation. Thus, if the item is complete with the dot at the end of the production, we can create a complete parse tree for this item by creating a new *Rule* node from the lhs of the production and all saved trees as its children.

The following invariant models the internal consistency between the item and the tree list. It makes sure that every parse tree is constructed correctly, that the leaves of the parse trees form the slice of w which can be derived from the production up to its dot, and that the roots of the parse trees are the symbols present in the production up to its dot. In a complete item, the roots of the trees must make up the production's lhs, as otherwise we would be constructing a parse tree, where the new *Rule* node does not represent a valid production.

$wf_parseItem :: ('n, 'a) \text{ parseItem} \Rightarrow \text{nat} \Rightarrow \text{bool}$
 $wf_parseItem \ x \ k =$
 $(wf_item \ (item \ x) \ k \wedge$
 $(\forall t \in \text{set} \ (trees \ x). \ parse_tree \ P \ t) \wedge$
 $fringes \ (rev \ (trees \ x)) = slice \ (from \ (item \ x)) \ k \ w \wedge$
 $map \ root \ (rev \ (trees \ x)) = take \ (dot \ (item \ x)) \ (rhs \ (prod \ (item \ x))))$

The parse trees in the list are stored in reverse order, since this makes insertions of new trees more convenient during the algorithm.

Similarly to the recognizer, we have created both a set-based and a list-based parser. We begin by introducing the set-based parser.

The overall structure is again similar to the recognizer using the *bins* function in conjunction with an inductive closure. So we need to augment the Earley operations to also construct the tree lists for every parse item.

$init_P :: ('n, 'a) \text{ parseItem set}$
 $init_P = \{(Item \ r \ 0 \ 0, []) \mid r \in P \wedge lhs \ r = S\}$

```

PredictP :: ('n, 'a) item ⇒ nat ⇒ ('n, 'a) parseItem set
PredictP x k = {(Item r 0 k, []) | r ∈ P ∧ next_sym_Nt x (lhs r)}

CompleteP ::
    ('n, 'a) parseItem set list ⇒ ('n, 'a) parseItem ⇒ ('n, 'a) parseItem set
CompleteP Bs y =
    (λ(p, t). (mv_dot p, Rule (lhs (prod (item y))) (rev (trees y)) # t)) '
    {x ∈ Bs ! from (item y) | next_sym_Nt (item x) (lhs (prod (item y)))}

ScanP :: ('n, 'a) parseItem set ⇒ nat ⇒ ('n, 'a) parseItem set
ScanP B k =
    {(mv_dot (item x), Sym (w ! k) # trees x) |
     x ∈ B ∧ next_symbol (item x) = Some (w ! k)}
    
```

In the *Init* and *Predict* operations, we start parsing a new substring, so we do not need to store any parse trees. The *Scan* operation directly parses a terminal symbol, which itself makes a complete parse tree, so we add it to the list of child parse trees. In the *Complete* operation, we have completed parsing a substring, so we construct a complete parse tree for this substring by creating a *Rule* node from the production's lhs and all of the child parse trees. This new tree is then added to all parse items, in which we advance the item's dot and thereby extend the derivation represented by the item. All of these operations fulfill the well-formed predicate. With these operations, we define an inductive set closure, *Close_P*, as in the recognizer case, and then use that to construct all parse bins again as we did prior.

```

binsP :: nat ⇒ ('n, 'a) parseItem set list
binsP 0 = [CloseP [] InitP]
binsP (k + 1) = (let bs = binsP k in bs @ [CloseP bs (ScanP (last bs) k)])
    
```

Lastly, we have to retrieve the full parse tree from a *final* item in the last bin.

```

get_parse_tree3 :: ('n, 'a) tree option
get_parse_tree3 =
    (let ts = SOME t. ∃ i. (i, t) ∈ last (binsP |w|) ∧ is_final i
     in if ∃ i t. (i, t) ∈ last (binsP |w|) ∧ is_final i
        then Some (Rule S (rev ts)) else None)
    
```

This top-level function returns a tree if a final item exists, and then constructs the complete parse tree from a completed parse item, just as the *Complete* operation. We need to use the Hilbert choice operator because multiple parse trees may exist for an item when the productions are ambiguous.

We now go on to prove correctness in two steps. First, we prove that any tree returned by our algorithm represents a valid derivation of *w* from *S* under the given productions. Second, we prove that our algorithm returns a parse tree if and only if *w* can be derived.

For the first claim, we have to prove that the parse bins we compute are well-formed, which then implies that every returned parse tree is valid. This requires showing that all

Earley operations return well-formed sets, which can then be extended to show that the closure and the bins are well-formed as well.

$$k \leq |w| \implies wf_parse_bins1 (bins_P k)$$

$$get_parse_tree3 = Some\ t \implies valid_parse_tree\ P\ w\ S\ t$$

To tackle the second claim, we use the fact that we have already proven that we recognize an item if and only if a derivation exists, which is equivalent to the existence of a final item in the last bin. Since the parser operations are only an augmentation of the recognizer operations, we do not change the items generated for each bin; we only add trees to them. Thus, we can prove that projecting the parse items in a bin to only their item parts yields the same bins as those computed by the recognizer algorithm. By extension, a final item is only present in the last parse bin if a derivation exists, and we only return a tree if this is the case, so our claim is true.

$$k \leq |w| \implies map\ ((\cdot)\ item)\ (bins_P k) = bins\ k$$

$$(\exists t. get_parse_tree3 = Some\ t) = (P \vdash [Nt\ S] \Rightarrow^* w)$$

6.1 List-based parser

We now proceed to construct an executable list-based parser. We mentioned that for ambiguous grammars, an item may have multiple valid parse trees representing its derivation. As the number of them is not linearly bounded, like the Earley items, we do not save every parse item, but only one per Earley item. This is different from the set version, where we did not specify such a constraint. So we will be working with parse item lists, where each Earley item appears only once, which will give us the same length bounds as in the recognizer case. To store such lists we do not write a new efficient item list for parse items, but instead we "unzip" the (n, a) *parseItem list* into a (n, a) *item list* and a (n, a) *tree list list*, where entries at the same index in both lists form a parse item in the original list. Then we can store the item list as an efficient item list, and the tree list list as is. The augmented parse item list *PIL* then looks like this.

$$parseIL :: (n, a)\ efficientItemList \times (n, a)\ tree\ list\ list$$

The parse item set represented by this data structure is obtained by zipping both lists.

$$set_{PIL} :: (n, a)\ parseIL \Rightarrow (n, a)\ parseItem\ set$$

$$set_{PIL}\ (il, ts) = set\ (zip\ (list\ il)\ ts)$$

So the semantic requirements for a valid parse item list are that the projection onto items is distinct, that the item list and tree list list have equal length, and that items and tree lists stored at the same index form a valid parse item. To capture the first two semantic requirements for the parse item list, we define a new invariant that reuses the efficient item list's invariant and ensures that the efficient item list and the tree list list are the same length for the zipping operation. The item list invariant already contains that the projected parse item list is distinct. The third semantic requirement is satisfied if the zipped set representation is well-formed.

$$\begin{aligned}
 inv_{PIL} &:: ('n, 'a) \text{ parseIL} \Rightarrow \text{bool} \\
 inv_{PIL} (il, ts) &= (inv_{IL} il \wedge |list\ il| = |ts|) \\
 wf_{PIL} &:: ('n, 'a) \text{ parseIL} \Rightarrow \text{nat} \Rightarrow \text{bool} \\
 wf_{PIL} pil\ k &= wf_parse_bin1 (set_{PIL} pil)\ k
 \end{aligned}$$

The operations on the parse item list are more or less the same as those for the basic item list. For the membership test, we only consider the Earley item part, while insertion boils down to insertion into the item list and the tree list. So the interesting functions are $isin_{PIL}$, $(\#_{PIL})$, and hd_{PIL} , which returns the parse item at the head of the list, which is no longer easily achievable through pattern matching.

$$\begin{aligned}
 isin_{PIL} &:: ('n, 'a) \text{ parseIL} \Rightarrow ('n, 'a) \text{ parseItem} \Rightarrow \text{bool} \\
 isin_{PIL} (il, _) (s, _) &= isin_{IL} il\ s \\
 (\#_{PIL}) &:: ('n, 'a) \text{ parseItem} \Rightarrow ('n, 'a) \text{ parseIL} \Rightarrow ('n, 'a) \text{ parseIL} \\
 (s, t) \#_{PIL} (il, ts) &= \\
 (if\ isin_{IL}\ il\ s\ then\ (il, ts)\ else\ (s \#_{IL}\ il, t \# ts)) \\
 hd_{PIL} &:: ('n, 'a) \text{ parseIL} \Rightarrow ('n, 'a) \text{ parseItem} \\
 hd_{PIL} (il, t \# _) &= (hd (list\ il), t)
 \end{aligned}$$

The other operations for the parse list are defined as those for the item list, using the adapted $isin_{PIL}$ and $(\#_{PIL})$ operations.

We again adapt the set-based *Init*, *Predict*, *Complete*, and *Scan* operations to work with lists. Then we again define a step function and closure as before, and finally use that to compute all parse bins. As a last step, we have to return the parse tree of a final item in the last bin, if such an item exists. This will then be a parse tree for the complete word.

$$\begin{aligned}
 get_parse_tree &:: ('n, 'a) \text{ parseItem list} \Rightarrow ('n, 'a) \text{ tree option} \\
 get_parse_tree [] &= \text{None} \\
 get_parse_tree (x \# xs) &= \\
 (if\ is_final\ (item\ x)\ then\ \text{Some}\ (Rule\ S\ (rev\ (trees\ x))) \\
 else\ get_parse_tree\ xs)
 \end{aligned}$$

Since we have introduced the constraint that the Earley items saved be distinct, we cannot prove the equivalence of the bins to the set-based parser. Instead, since we are reusing the operations of the efficient item list, we can prove that those are the same in the parsing and recognition algorithms. In conjunction with proving that the Earley operations restricted to items are also the same we, can prove that the projection of parse bins onto the items is the same as the bins computed for the recognizer. As examples, we show equality of the *Predict* operation and the *bins* function:

$$\begin{aligned}
 map\ item\ (Predict_{P,L}\ s\ k) &= Predict_L\ s\ k \\
 k \leq |w| \implies map\ (map\ item)\ (bins_{PIL}\ k) &= bins_{IL}\ k
 \end{aligned}$$

Additionally, we show that the computed parse bins are well-formed.

We can then show, similarly to the set algorithm, that our algorithm returns a parse tree only if the word is in the language, and that any parse tree it returns is valid under the productions. Thus, our algorithm returns only valid parse trees and only if the word is in the language.

Theorem 5. $(\exists t. \text{get_parse_tree} (\text{last} (\text{bins}_{PIL} |w|)) = \text{Some } t) \iff w \in \text{lang } ps \ S$
 $\text{get_parse_tree} (\text{last} (\text{bins}_{PIL} |w|)) = \text{Some } t \implies \text{valid_parse_tree } P \ w \ S \ t$

As a simple corollary, we get that if the grammar is unambiguous, we return the singular correct tree, and that the set-based and list-based parsers are equal.

Corollary 1. $\text{unambiguous} (CfG \ P \ S) \implies \text{valid_parse_tree } P \ w \ S \ t = (\text{get_parse_tree} (\text{last} (\text{bins}_{PIL} |w|)) = \text{Some } t)$

$\llbracket G = CfG \ P \ S; \text{unambiguous } G \rrbracket \implies \text{get_parse_tree3} = \text{get_parse_tree} (\text{last} (\text{bins}_{PIL} |w|))$

6.2 Runing Time Analysis of Parsing Algorithm

The running time analysis works very similarly to the one for the recognizer. One big difference is that in the recognizer, we could make assumptions about the size of all well-formed bins, whereas the *wf_parse_bin* predicate does not allow for the same bounds. We only have the same size bounds if the bin was computed using a *PIL* that fulfills its invariant. So we have to introduce an assumption about the sizes of these bins, where we do not have any other information about them, for example, the *Complete* operation where we only assume to have a list of well-formed bins given. In the end, this bound is resolved in the analysis of the *bins_{PIL}* method, where we can finally show that all bins are computed using a *PIL* and have the same bound as in the recognizer. For the time bound of the *Complete* operation, we introduce the assumption that the bin size is bounded by C ($|Bs ! \text{from} (item \ pi)| \leq C$) and obtain this time bound, where we do not show other assumptions that need to hold, for readability.

$$|Bs ! \text{from} (item \ pi)| \leq C \implies T_{\text{Complete}_{P,L}} \ Bs \ pi \leq |Bs| + (2 * K * K + 2 * K + 5) * C + 2$$

Overall, the time bounds are very similar to those of the recognizer, since the parsing functions follow the same scheme and we reuse item list functions. One difference appears in the insert operation of the parse item list, since both the parse item list and the item list perform inclusion checks during insertion. One of these could be removed for optimization, but was left in, as it does not change the overall time bound. If we compare the bounds of both operations, we see that they differ by $T_{nth_{IL}} (\text{from } x) + L * (K + 1) + 1$, which is the bound for *isin_{IL}*.

$$\begin{aligned} T_{\#_{IL}} \ x \ il &\leq 3 * T_{nth_{IL}} (\text{from } x) + L * (K + 1) + 1 \\ T_{\#_{PIL}} \ x \ pil &\leq 4 * T_{nth_{IL}} (\text{from} (item \ x)) + 2 * L * (K + 1) + 2 \end{aligned}$$

This difference then affects all other parse list running times as well, as they all use (*#_{PIL}*).

Another difference is that the *Parse_Complete_L* operation takes more time than the recognizer operation, since it has to reverse the tree list attached to the item to build a valid parse tree for the completed item. The running time of the inefficient list reversal is $(2 * K * K + 2 * K + 5)$ and may be incurred for every item in the bin. Again, we could use a more efficient list reversal using an accumulator, but it does not change the overall asymptotic running time, as both are constant in the size of the word.

The end result is that while the asymptotic running time stays the same as in the recognizer case, the prefactors change.

$$\begin{aligned}
 & T_{bins_{PL}} \ k \\
 & \leq (k + 2)^3 * C3 + (k + 2)^2 * C4 + (k + 2) * 3 + C5 + \\
 & \quad (k + 2)^3 * T_{nth_{IL}} \ k * C6 + (k + 2)^2 * T_{nth_{IL}} \ k * C7 \\
 & C3 = 10 * (L + 1)^3 * (K + 1)^4 \\
 & C4 = 10 * (L + 1)^2 * (K + 1)^3 \\
 & C5 = 10 * (L + 1)^3 * (K + 1)^3 + 25 * (L + 1)^2 * (K + 1)^2 * T_{nth_{IL}} \ 0 \\
 & C6 = 17 * (L + 1)^2 * (K + 1)^2 \\
 & C7 = L * (K + 1) * 4
 \end{aligned}$$

Combining this with the running time for the extraction of the final tree gives us:

$$\begin{aligned}
 & T_{get_parse_tree_w \ ps \ S \ w0} \\
 & \leq (|w| + 2)^3 * C3 + (|w| + 2)^2 * C4 + (|w| + 2) * C8 + C5 + \\
 & \quad (|w| + 2)^3 * T_{nth_{IL}} \ |w| * C6 + (|w| + 2)^2 * T_{nth_{IL}} \ |w| * C7 \\
 & C8 = 6 * (L + 1) * (K + 1)^2
 \end{aligned}$$

7 Related Work

Lasser formalizes two parsers based on the LL(1) and All(*) algorithm approaches using the Coq proof assistant [8]. They prove soundness and completeness of both parsers, as well as termination, as long as the grammars are LL(1) or non-left-recursive, respectively. Instead of proving a formal running time bound they measure the parsers' practical performance empirical tests. Finally, they augment their All(*) parser to handle grammars with semantic predicates, which are checked during parsing to ensure the input is semantically well-formed.

Firsov and Uustalu verify a parser based on the CYK algorithm in Agda [4]. Their work encompasses correctness and termination proofs, assuming the grammar is in Chomsky-Normal Form.

Koprowski and Binsztok verify a parser for parsing expression grammars (PEGs) in Coq [7]. They prove correctness and termination by first inspecting the grammar and rejecting those that would not lead to termination. On running time, they mention that their algorithm may take exponential time on grammars that are not LL(k). A solution to this problem that trades a linear running time for higher space requirements is the packrat parsing approach using memoization. A formal verification of such a PEG parser has been performed by Blaudeau and Shankar [2] in PVS.

Another approach is used by Jordan et al., who write a validator for LR(1) parsers in Coq [6]. The validator takes a CFG and an otherwise generated parser and validates that the parser works correctly for the given grammar. One aspect that is not verified is that the given parser terminates for invalid input. This validation approach has the benefit of verifying the output of optimized, already existing parser generators and could thus be incorporated into those, guaranteeing correctness. Thus, this approach can be used with a variety of parser generators, without having to prove correctness of each one individually.

8 Conclusion

In this thesis, we refined the prior work done by Nipkow and Rau on the Earley algorithm into an executable list-based recognizer. The recognizer was proven correct for all epsilon-free grammars and could be adapted to work on those as well, as outlined in [section 3.4](#). We also prove termination in all cases. For running time guarantees, we implemented the $(\text{'n'}, \text{'a'})$ *efficientItemList* data structure proposed by Earley. Using a parameter for the random-access time of the partitions in this data structure, we conducted an asymptotic running time analysis, verifying a cubic running time under constant access time. Our list-based implementation achieves only an $O(n^4)$ running time instead.

We then verified both a set-based and a list-based parser using this algorithm. Both algorithms are correct and always terminate for epsilon-free grammars, and were proven to be equivalent if the grammar is unambiguous. In the case of an ambiguous grammar, our algorithm returns only one random parse tree. The running time analysis resulted in the same time bounds as the recognizer.

For future work, we see a possibility in replacing the list used in the $(\text{'n'}, \text{'a'})$ *efficientItemList* datatype by an actual array implementation, as this would guarantee the best running time bounds for this algorithm. Another avenue is to formalize the treatment of grammars containing epsilon productions using one of the approaches outlined in [section 3.4](#).

Earley also presents sets of languages for which the algorithm has a better running time bound, which could be of interest. Unambiguous reduced grammars could have quadratic running time because the set-difference operation can be omitted. There are also grammars for which the size of each Earley bin is bounded by a constant, leading to linear running time. Earley mentions that most LR(k) grammars fall under this, but does not give a characterization of exactly which. Fully characterizing this set of grammars and verifying their linear running time is also an interesting direction.

Lastly, CFGs can be extended to give certain productions higher priority when multiple apply, resolving ambiguity. This feature is used in many programming languages as well, so an extension to these would be great, but would require a full extension of the derivation infrastructure for CFGs.

Bibliography

- [1] Functional Data Structures and Algorithms: A Proof Assistant Approach. ACM (2025), <https://doi.org/10.1145/3731369>
- [2] Blaudeau, C., Shankar, N.: A verified packrat parser interpreter for parsing expression grammars. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. pp. 3–17. POPL '20, ACM (Jan 2020), <https://doi.org/10.1145/3372885.3373836>
- [3] Earley, J.: An efficient context-free parsing algorithm. Communications of the ACM **13**(2), 94–102 (Feb 1970). <https://doi.org/10.1145/362007.362035>
- [4] Firsov, D., Uustalu, T.: Certified cyk parsing of context-free languages. Journal of Logical and Algebraic Methods in Programming **83**(5–6), 459–468 (2014), <https://doi.org/10.1016/j.jlamp.2014.09.002>
- [5] Jones, C.B.: Formal development of correct algorithms: An example based on earley’s recogniser. ACM SIGACT News (14), 150–169 (Jan 1972). <https://doi.org/10.1145/942580.807083>
- [6] Jourdan, J.H., Pottier, F., Leroy, X.: Validating LR(1) Parsers, pp. 397–416. Springer Berlin Heidelberg (2012), https://doi.org/10.1007/978-3-642-28869-2_20
- [7] Koprowski, A., Binsztok, H.: Trx: A formally verified parser interpreter. Logical Methods in Computer Science **Volume 7, Issue 2** (2011), [https://doi.org/10.2168/LMCS-7\(2:18\)2011](https://doi.org/10.2168/LMCS-7(2:18)2011)
- [8] Lasser, S.: Formal Verification of Top-Down Parser Interpreters. Tufts University (2022)
- [9] Nipkow, T., Rau, M.: A Note on Proofs of Earley’s Recognizer, pp. 45–55. Springer Nature Switzerland (2024), https://doi.org/10.1007/978-3-031-66673-5_3